IgH EtherCAT 主站开发案例

Revision History

Draft Date	Revision No.	Description
2024/10/22	V1.1	1. 更新“案例测试”小节。
2024/06/25	V1.0	1. 初始版本。

目　　录

前　言 3
1　简介 5
1.1　Real-time Edge 5
1.2　IgH EtherCAT 简介 5
2　案例说明 6
3　案例测试 7
3.1　电机正转 15
3.1.1　SCHED_FIFO 模式（空载状态） 15
3.1.2　SCHED_FIFO 模式（满负荷状态） 15
3.1.3　SCHED_DEADLINE 模式（空载状态） 16
3.1.4　SCHED_DEADLINE 模式（满负荷状态） 17
3.1.5　统计结果分析 18
3.2　电机反转 20
4　案例编译 20
4.1　内核镜像、内核模块编译 20
4.2　Igh EtherCAT 主站程序编译 24
4.3　igh_ethercat_dc_motor 案例编译 31
5　关键代码说明 32
更多帮助 36

前 言

本文档主要演示 TLIMX8MP-EVM 评估板基于 IgH EtherCAT 控制伺服电机。本文档适用开发环境：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

虚拟机：VMware15.5.5

Linux 开发环境：Ubuntu20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71

Linux SDK：Real-Time Edge Software 2.5

伺服驱动器：台达 ASD-A2-0121-E

伺服电机：台达 ECMA-C10401GS

我司提供的 IgH EtherCAT 主站开发案例 igh_ethercat_dc_motor 位于产品资料“4-软件资料\Demo\rt-linux-demos\”目录下，主要包含 bin、src、igh_ethercat、patch 等文件夹，具体说明如下。

表 1

案例目录		目录说明
bin	image	存放内核镜像
	modules	存放内核模块
	igh_ethercat_dc_motor	存放 igh_ethercat_dc_motor 应用程序镜像
src		存放 igh_ethercat_dc_motor 应用程序源码
igh_ethercat	src	存放 IgH EtherCAT 开发包源码，可用于生成 EtherCAT 驱动和 igh_ethercat_dc_motor 应用程序所需的头文件、链接库文件
	images	存放 IgH EtherCAT 镜像
patch		存放补丁文件，用于编译驱动

本文档测试需使用案例专用的 Linux-RT 内核镜像和内核模块，位于产品资料目录“4-软件资料\Demo\rt-linux-demos\igh_ethercat_dc_motor\bin\”目录下的 image、modules

文件夹，请参考《Linux 系统启动卡制作及固化》文档，替换制卡工具包中对应的文件，然后进行 Linux 系统启动卡制作。

1 简介

1.1 Real-time Edge

Real-time Edge 软件支持 i.MX 和 Layerscape 产品的实时应用，并且可以轻松地与这些产品的 Yocto SDK 集成。Real-time Edge 软件支持面向低延迟应用的 Preempt-RT、裸机框架和采用 Cortex A 或 Cortex M 内核的实时操作系统(RTOS)，从而实现实时功能。Real-time Edge 软件还支持 IEEE1588 PTP 网络同步、时间敏感网络(实时以太网)、EtherCAT master、CANOpen 和 OPC-UA 的实时协议和堆栈。

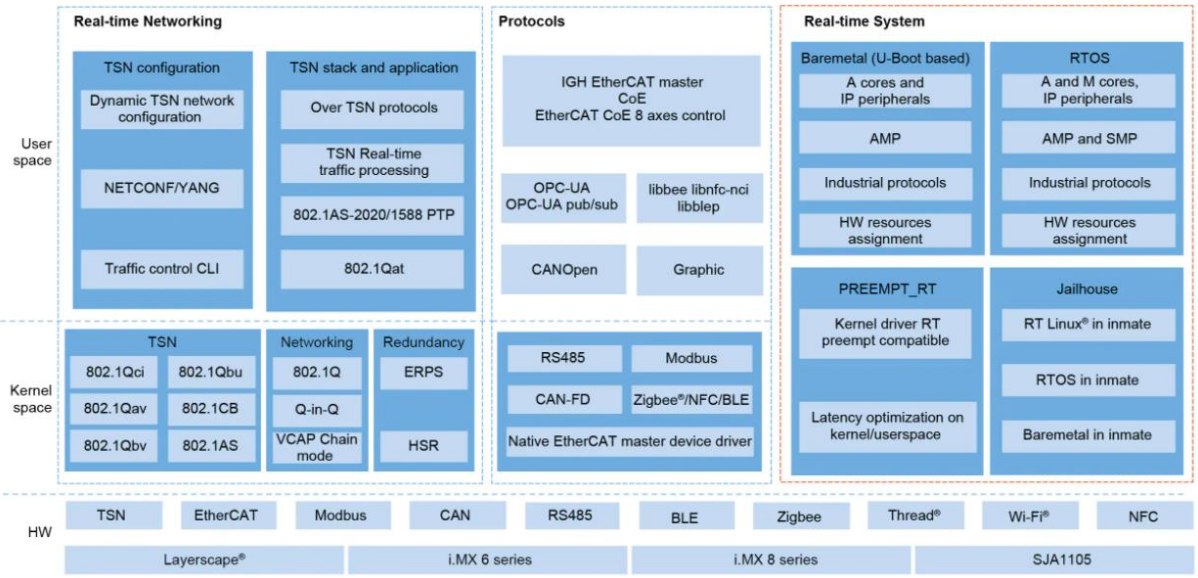

图 1

1.2 IgH EtherCAT 简介

IgH EtherCAT 为运行于 Linux 系统的免费开源 EtherCAT 主站程序，框架如下所示。

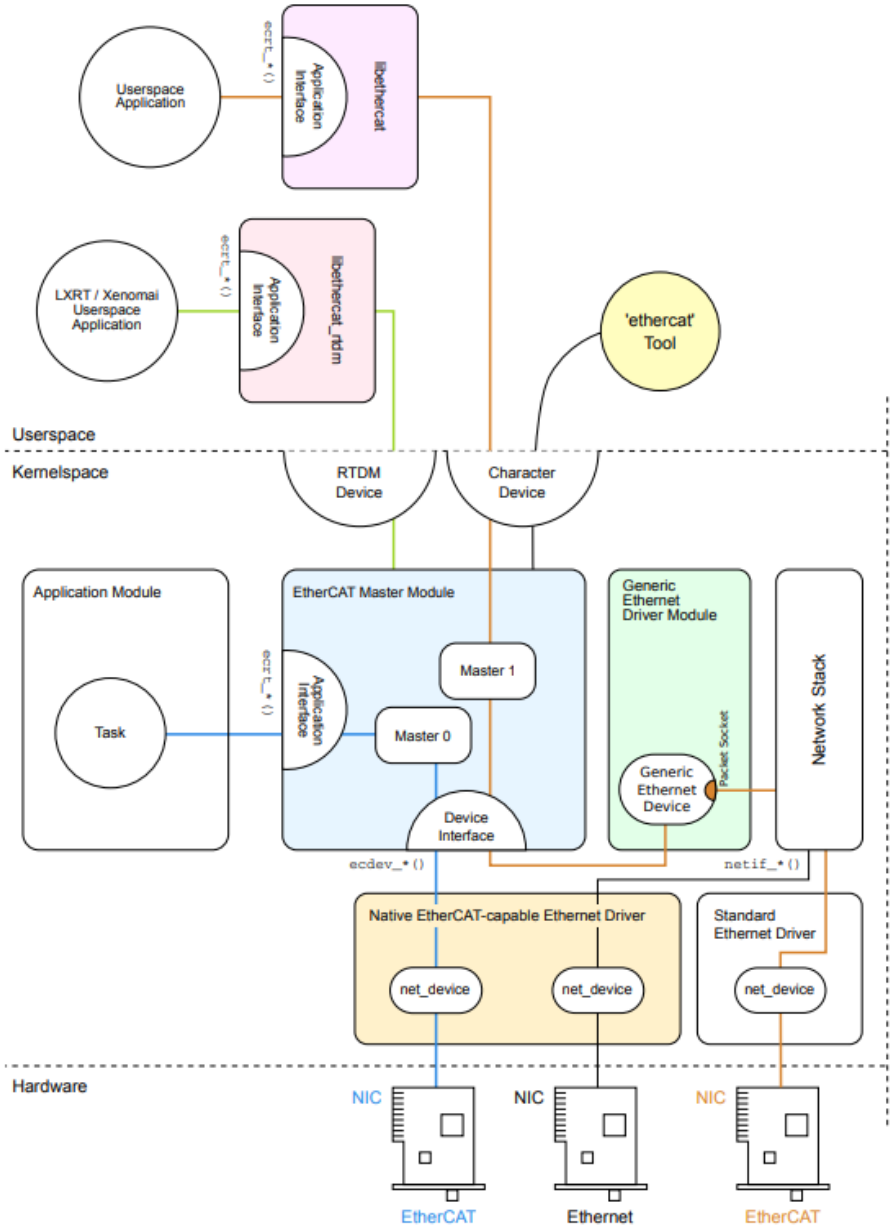

Figure 2.1: Master Architecture

图 2

IgH EtherCAT 主站通过构建 Linux 字符设备，应用程序通过对字符设备的访问实现与 EtherCAT 主站模块的通信。

IgH EtherCAT 开发包提供 EtherCAT 工具，该工具提供各种可在 Linux 用户层运行的命令，可直接实现对从站的访问和设置，如设置从站地址、显示总线配置、显示 PDO 数据、读写 SDO 参数等。

2 案例说明

案例功能：EtherCAT 通讯周期时间为 1ms，控制伺服电机正转和反转，并通过串口循环打印 EtherCAT 通讯周期时间的最大值和最小值。

（1） 正转：伺服电机目标速度从 0 加速到 10000，当达到 10000 速度后，控制伺服电机减速至 0，循环运行。

（2） 反转：伺服电机目标速度从 0 加速到-10000，当达到-10000 速度后，控制伺服电机减速至 0，循环运行。

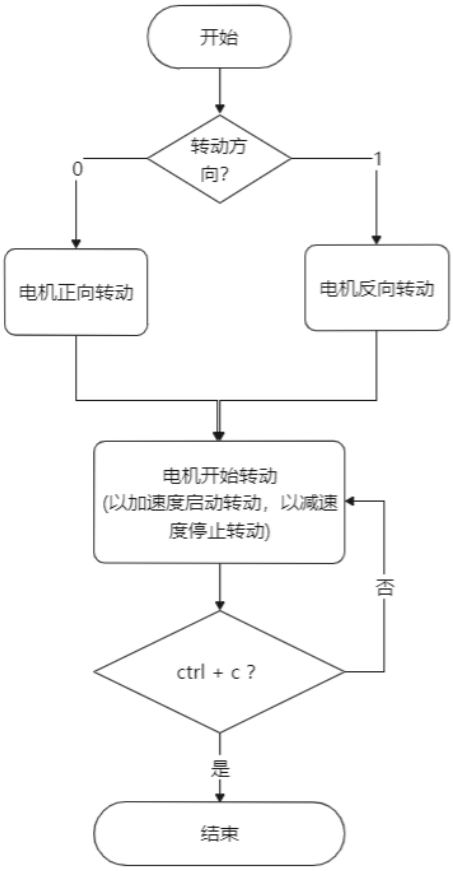

图 3

3 案例测试

在本案例测试过程中，程序均在隔离 CPU 环境下运行，并分别采用 SCHED_FIFO 和 SCHED_DEADLINE 两种进程调度策略，在空载状态、满负荷状态下持续运行 12 小时。

请按下图所示使用网线连接评估板 ETH1 网口和伺服驱动器 A 的 IN 网口，将伺服驱动器 A 的 OUT 网口使用网线连接至伺服驱动器 B 的 IN 网口。

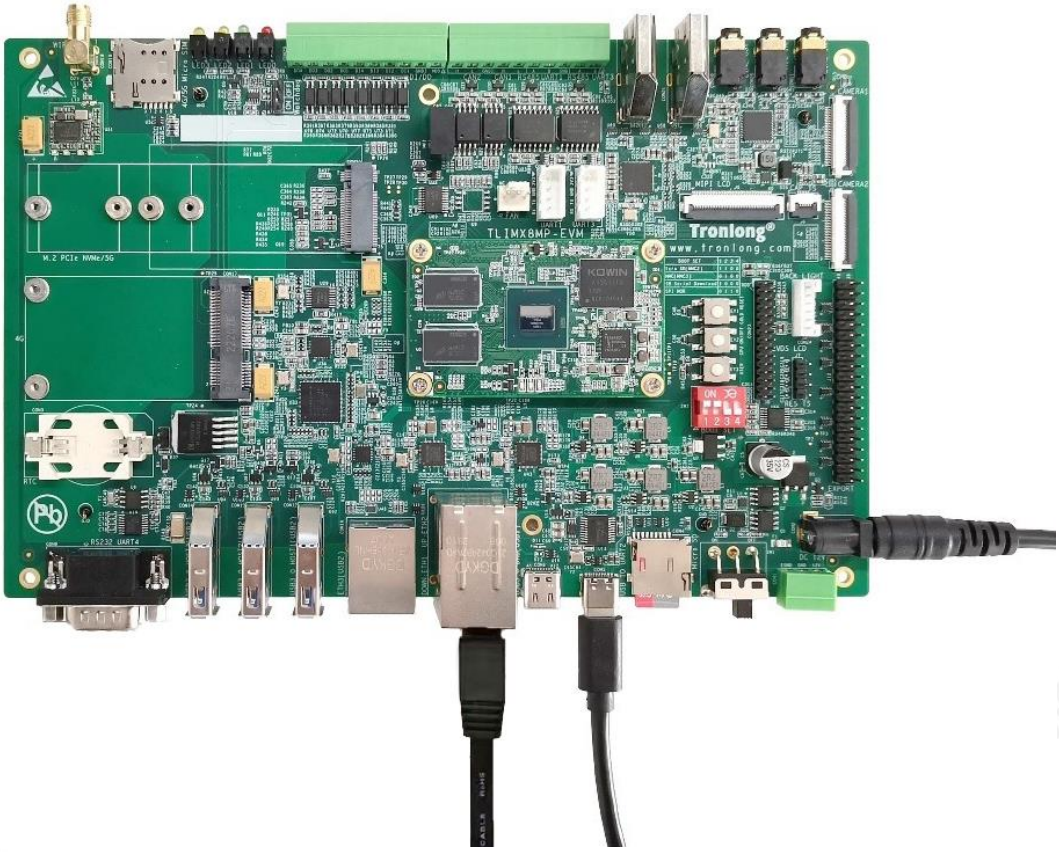
TL IMX8MP-EVM
Tronlong®
www.tronlong.com

图 4

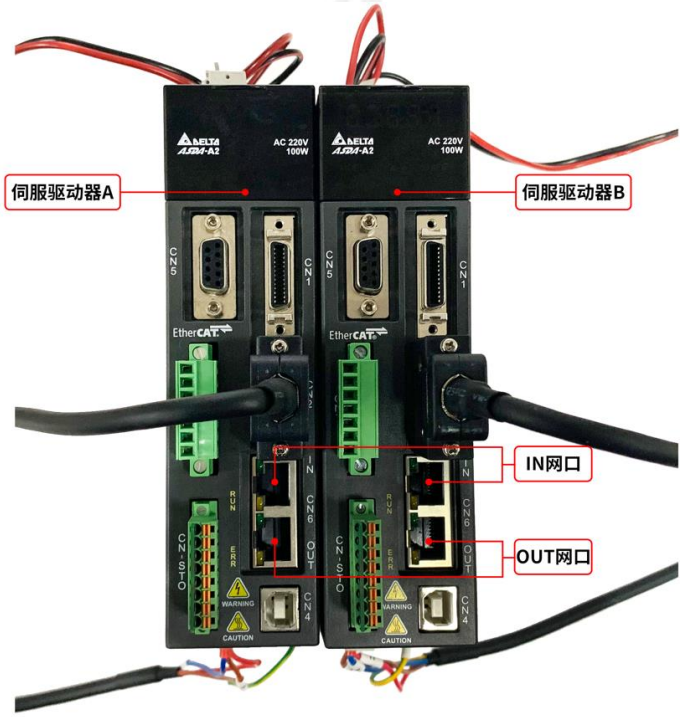
伺服驱动器A
伺服驱动器B
IN网口
OUT网口
AC 220V
100W
EtherCAT

图 5

评估板上电时，按下空格键，进入 U-Boot 命令行。执行如下命令配置隔离 CPU，本次以隔离 CPU3 为例进行演示。

备注：isolcpus 参数的作用为修改环境变量隔离 CPU3，降低系统上运行的其他程序对隔离 CPU 的延迟影响，保证任务在 CPU3 核心上运行。

U-Boot# setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} cpuidle.off=1 isolcpus=3"

U-Boot# saveenv

U-Boot# reset

```
u-boot=>
u-boot=> setenv mmcargs "setenv bootargs ${jh_clk} ${mcore_clk} console=${console} root=${mmcroot} cpuidle.off=1 isolcp
us=3"
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 6

将评估板重启，进入系统后执行如下命令，查看环境变量确认已隔离 CPU3 核心。

Target# cat /proc/cmdline

```
root@IMX8MP-Tronlong:~# cat /proc/cmdline
console=ttymxc1,115200 root=/dev/mmcblk1p2 rootwait rw cpuidle.off=1 isolcpus=3
root@IMX8MP-Tronlong:~#
```

图 7

为便于测试，我司提供经验证的 IgH EtherCAT 主站程序为案例"igh_ethercat\images\"目录下的 ethercat-[版本号]-[Git 序列号].tar.gz 压缩包，将其拷贝至评估板文件系统任意目录下。版本号、Git 序列号请以实际情况为准。将案例 bin 目录下的 igh_ethercat_dc_motor 可执行文件拷贝至评估板文件系统。

执行如下命令，解压 IgH EtherCAT 主站程序压缩包，得到__install 文件夹。

Target# tar -xf ethercat-1.5.2-g418dc4b.tar.gz

```
root@IMX8MP-Tronlong:~# ls
ethercat-1.5.2-g418dc4b.tar.gz
root@IMX8MP-Tronlong:~# tar -xf ethercat-1.5.2-g418dc4b.tar.gz
tar: __install/etc/init.d/ethercat: time stamp 2024-03-08 10:13:20 is 58671986.835940375 s in the future
tar: __install/etc/init.d: time stamp 2024-03-08 10:13:20 is 58671986.83568925 s in the future
tar: __install/etc/ethercat.conf: time stamp 2024-03-08 10:13:20 is 58671986.835332375 s in the future
tar: __install/etc/sysconfig/ethercat: time stamp 2024-03-08 10:13:20 is 58671986.834986625 s in the future
tar: __install/etc/sysconfig: time stamp 2024-03-08 10:13:20 is 58671986.8348765 s in the future
tar: __install/etc: time stamp 2024-03-08 10:13:20 is 58671986.834751625 s in the future
tar: __install/lib/libethercat.la: time stamp 2024-03-08 10:13:20 is 58671986.834417 s in the future
tar: __install/lib/systemd/system/ethercat.service: time stamp 2024-03-08 10:13:20 is 58671986.833928 s in the future
tar: __install/lib/systemd/system: time stamp 2024-03-08 10:13:20 is 58671986.833817125 s in the future
tar: __install/lib/systemd: time stamp 2024-03-08 10:13:20 is 58671986.83370225 s in the future
tar: __install/lib/libethercat.so.1.0.0: time stamp 2024-03-08 10:13:20 is 58671986.8314505 s in the future
tar: __install/lib/libethercat.so: time stamp 2024-03-08 10:13:20 is 58671986.831192125 s in the future
tar: __install/lib/libethercat.a: time stamp 2024-03-08 10:13:20 is 58671986.825608375 s in the future
tar: __install/lib/libethercat.so.1: time stamp 2024-03-08 10:13:20 is 58671986.825342625 s in the future
tar: __install/lib: time stamp 2024-03-08 10:13:20 is 58671986.825240125 s in the future
tar: __install/bin/ethercat: time stamp 2024-03-08 10:13:20 is 58671986.669569625 s in the future
tar: __install/bin: time stamp 2024-03-08 10:13:20 is 58671986.669364625 s in the future
tar: __install/include/ectty.h: time stamp 2024-03-08 10:13:20 is 58671986.668821375 s in the future
tar: __install/include/ecrt.h: time stamp 2024-03-08 10:13:20 is 58671986.6678135 s in the future
tar: __install/include: time stamp 2024-03-08 10:13:20 is 58671986.667637375 s in the future
tar: __install/modules/ec_master.ko: time stamp 2024-03-08 10:14:12 is 58672038.637549625 s in the future
tar: __install/modules/ec_fec.ko: time stamp 2024-03-08 10:14:17 is 58672043.631015 s in the future
tar: __install/modules/ec_generic.ko: time stamp 2024-03-08 10:14:06 is 58672032.629899 s in the future
tar: __install/modules: time stamp 2024-03-08 10:14:17 is 58672043.629725125 s in the future
tar: __install/sbin/ethercatctl: time stamp 2024-03-08 10:13:20 is 58671986.629209375 s in the future
tar: __install/sbin: time stamp 2024-03-08 10:13:20 is 58671986.628997 s in the future
tar: __install: time stamp 2024-03-08 10:14:01 is 58672027.628842125 s in the future
root@IMX8MP-Tronlong:~# ^C
root@IMX8MP-Tronlong:~# ls
__install  ethercat-1.5.2-g418dc4b.tar.gz
root@IMX8MP-Tronlong:~#
```

图 8

执行如下命令，将.ko 文件拷贝至内核模块对应版本的目录。

Target# cp __install/modules/ec_*.ko /lib/modules/5.15.71-rt51-g56cc3dc8d258/

Target# sync

Target# depmod

```
root@IMX8MP-Tronlong:~# cp __install/modules/ec_*.ko /lib/modules/5.15.71-rt51-g56cc3dc8d258/
root@IMX8MP-Tronlong:~# sync
root@IMX8MP-Tronlong:~# depmod
root@IMX8MP-Tronlong:~#
```

图 9

执行如下命令，将 sysconfig 文件夹拷贝至 etc 目录。

Target# cp -r __install/etc/sysconfig/ /etc/

Target# sync

```
root@IMX8MP-Tronlong:~# cp -r __install/etc/sysconfig/ /etc/
root@IMX8MP-Tronlong:~# sync
root@IMX8MP-Tronlong:~#
```

图 10

ethercat 工具位于"/home/root/__install/bin/"下，可调用该工具进行从站的扫描和配置。

执行如下命令，查看 ethercat 工具相关使用方法。

Target# /home/root/__install/bin/ethercat –help

```
root@IMX8MP-Tronlong:~# /home/root/__install/bin/ethercat --help
Usage: ethercat <COMMAND> [OPTIONS] [ARGUMENTS]

Commands (can be abbreviated):
  alias      Write alias addresses.
  config     Show slave configurations.
  cstruct    Generate slave PDO information in C language.
  data       Output binary domain process data.
  debug      Set the master's debug level.
  domains    Show configured domains.
  download   Write an SDO entry to a slave.
  eoe        Display Ethernet over EtherCAT statictics.
  foe_read   Read a file from a slave via FoE.
  foe_write  Store a file on a slave via FoE.
  graph      Output the bus topology as a graph.
  master     Show master and Ethernet device information.
  pdos       List Sync managers, PDO assignment and mapping.
  reg_read   Output a slave's register contents.
  reg_write  Write data to a slave's registers.
  rescan     Rescan the bus.
  sdos       List SDO dictionaries.
  sii_read   Output a slave's SII contents.
  sii_write  Write SII contents to a slave.
  slaves     Display slaves on the bus.
  soe_read   Read an SoE IDN from a slave.
  soe_write  Write an SoE IDN to a slave.
  states     Request application-layer states.
  upload     Read an SDO entry from a slave.
  version    Show version information.
  xml        Generate slave information XML.

Global options:
  --master  -m <master>  Comma separated list of masters
                         to select, ranges are allowed.
                         Examples: '1,3', '5-7,9', '-3'.
                         Default: '-' (all).
  --force   -f           Force a command.
  --quiet   -q           Output less information.
  --verbose -v           Output more information.
  --help    -h           Show this help.

Numerical values can be specified either with decimal (no
prefix), octal (prefix '0') or hexadecimal (prefix '0x') base.

Call 'ethercat <COMMAND> --help' for command-specific help.

Send bug reports to fp@igh-essen.com.
root@IMX8MP-Tronlong:~#
```

图 11

评估板重新上电，系统会自动加载默认的 ec_fec.ko、ec_master.ko 驱动，需执行如下命令进行查看并将其卸载。

Target# lsmod

Target# rmmod ec_fec

Target# rmmod ec_master

```
root@IMX8MP-Tronlong:~# lsmod
Module                  Size  Used by
fsl_jr_uio             20480  0
caam_jr               180224  0
caamkeyblob_desc       16384  1 caam_jr
caamhash_desc          16384  1 caam_jr
caamalg_desc           40960  1 caam_jr
crypto_engine          16384  1 caam_jr
authenc                16384  1 caam_jr
libdes                 24576  1 caam_jr
overlay               122880  0
crct10dif_ce           20480  1
ec_fec                 45056  0
ec_master             290816  1 ec_fec
imx8_media_dev         20480  0
snd_soc_fsl_aud2htx    16384  0
snd_soc_fsl_easrc      45056  0
snd_soc_fsl_asrc       40960  1 snd_soc_fsl_easrc
dw_hdmi_cec            16384  0
snd_soc_fsl_sai        40960  2
ov4689                 28672  0
rtc_ds1307             32768  0
fec                    65536  0
caam                   28672  1 caam_jr
secvio                 20480  0
error                  24576  7 caamalg_desc,secvio,caamkeyblob_desc,caamhash_desc,caam,caam_jr,
fsl_jr_uio
flexcan                32768  0
can_dev                36864  1 flexcan
imx_dsp_rproc          20480  0
pwm_fan                20480  0
fuse                  131072  1
root@IMX8MP-Tronlong:~# rmmod ec_fec
root@IMX8MP-Tronlong:~#
```

图 12

```
root@IMX8MP-Tronlong:~# rmmod ec_fec
root@IMX8MP-Tronlong:~# rmmod ec_master
[   56.338881] EtherCAT: Master module cleaned up.
root@IMX8MP-Tronlong:~#
```

图 13

执行如下命令，查询评估板网卡物理地址。

Target# ifconfig

```
root@IMX8MP-Tronlong:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet6 fe80::ec89:74ff:feac:dbdc  prefixlen 64  scopeid 0x20<link>
        ether ee:89:74:ac:db:dc  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 16  dropped 0  overruns 0  frame 16
        TX packets 25  bytes 4368 (4.2 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth1: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 1e:bd:85:ca:d3:5d  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 61

eth2: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 00:e0:99:8a:e4:0a  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 100  bytes 8953 (8.7 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 100  bytes 8953 (8.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@IMX8MP-Tronlong:~#
```

图 14

执行如下命令，加载驱动 ec_master.ko。命令中"ee:89:74:ac:db:dc"为评估板网卡物理地址，请根据实际情况修改。

备注：由于上述已执行 depmod 命令产生模块依赖的映射文件，因此加载"__install/modules/"文件夹或"/lib/modules/5.15.71-rt51-[Git 序列号]/"文件夹下的驱动文件均可。

Target# insmod __install/modules/ec_master.ko main_devices=ee:89:74:ac:db:dc

```
root@IMX8MP-Tronlong:~# insmod __install/modules/ec_master.ko main_devices=ee:89:74:ac:db:dc
[  312.411409] ec_master: loading out-of-tree module taints kernel.
[  312.415885] EtherCAT: Master driver 1.5.2 2eff7c993a63
[  312.416214] EtherCAT: 1 master waiting for devices.
root@IMX8MP-Tronlong:~#
```

图 15

执行如下命令，使能 IgH EtherCAT master。

Target# /home/root/__install/etc/init.d/ethercat start

```
root@IMX8MP-Tronlong:~# /home/root/__install/etc/init.d/ethercat start
Starting EtherCAT master 1.5.2  done
root@IMX8MP-Tronlong:~#
```

图 16

执行如下命令，卸载系统自动加载的 fec 驱动。

Target# rmmod fec

```
root@IMX8MP-Tronlong:~# rmmod fec
root@IMX8MP-Tronlong:~#
```

图 17

执行如下命令，加载驱动 ec_fec.ko。

Target# insmod /lib/modules/5.15.71-rt51-g56cc3dc8d258/ec_fec.ko

```
root@IMX8MP-Tronlong:~# rmmod fec
root@IMX8MP-Tronlong:~# insmod /lib/modules/5.15.71-rt51-g56cc3dc8d258/ec_fec.ko
[  782.831089] pps pps0: new PPS source ptp0
[  782.856848] ec_fec 30be0000.ethernet eth2: registered PHC device 0
root@IMX8MP-Tronlong:~# [  783.239790] YT8521SH Gigabit Ethernet 30be0000.ethernet-1:00: attache
d PHY driver (mii_bus:phy_addr=30be0000.ethernet-1:00, irq=POLL)
[  785.321759] ec_fec 30be0000.ethernet eth2: Link is Up - 100Mbps/Full - flow control off
[  785.321772] EtherCAT WARNING: ecdev_set_link() called with null device!
[  785.321803] IPv6: ADDRCONF(NETDEV_CHANGE): eth2: link becomes ready

root@IMX8MP-Tronlong:~#
```

图 18

执行如下命令,添加动态链接库路径，并修改内核信息打印级别，防止打印信息干扰。

Target# export LD_LIBRARY_PATH=/home/root/__install/lib:${LD_LIBRARY_PATH}

Target# echo 1 4 1 7 > /proc/sys/kernel/printk

```
root@IMX8MP-Tronlong:~# export LD_LIBRARY_PATH=/home/root/__install/lib:${LD_LIBRARY_PATH}
root@IMX8MP-Tronlong:~# echo 1  4  1  7 > /proc/sys/kernel/printk
root@IMX8MP-Tronlong:~#
```

图 19

执行以下命令，查看程序运行参数。

Target# ./igh_ethercat_dc_motor --help

```
root@IMX8MP-Tronlong:~# ./igh_ethercat_dc_motor --help
Usage: igh_ethercat_dc_motor [options]

Options:
 -d | --direction          direction of rotation (foreward: 0, reverse: 1)
 -s | --schedule           selection of mode (SCHED_FIFO:0,SCHED_DEADLINE:1 )
 -v | --version            Version Info.
 --help                    Show this message.

e.g. :
   ./igh_ethercat_dc_motor -d 0
   ./igh_ethercat_dc_motor -d 0 -s 0
   ./igh_ethercat_dc_motor -d 0 -s 1 [<sched_runtime> <sched_deadline> <sched_period>]

root@IMX8MP-Tronlong:~#
```

图 20

参数解释

（1） -d：指定电机转向，0：正转，1：反转；

（2） -v：打印程序版本信息；

（3） -s：选择进程调度策略模式。程序执行时未使用-s 参数或-s 参数值为 0，进程调度策略默认为 SCHED_FIFO；若使用-s 参数且参数值为 1，进程调度策略为 SCHED_DEADLINE，后面未添加参数值[xxx xxx xxx]则使用默认参数值，其中 sched-runtime 值表示运行时间、sched-deadline 值表示相对的期限、sched-period 值表示周期，单位均为 ns；

（4） -help：查看程序运行参数。

3.1 电机正转

3.1.1 SCHED_FIFO 模式（空载状态）

该测试状态为程序运行在隔离 CPU，程序进程调度策略设置为 SCHED_FIFO，进程优先级设置为最高，空载状态测试 12 个小时。

执行以下命令，控制两台伺服电机同时正转。

Target# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 0 > log.txt &

```
root@IMX8MP-Tronlong:~# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 0 > log.txt &
[1] 915
root@IMX8MP-Tronlong:~#
```

图 21

打印信息"915"为进程号，请根据实际进程号，执行如下命令，确认进程调度策略。

Target# chrt -p 915

```
root@IMX8MP-Tronlong:~# chrt -p 915
pid 915's current scheduling policy: SCHED_FIFO
pid 915's current scheduling priority: 99
root@IMX8MP-Tronlong:~#
```

图 22

如需停止运行后台程序，请执行如下命令。

Target# kill -9 915

```
root@IMX8MP-Tronlong:~# kill -9 915
root@IMX8MP-Tronlong:~#
[1]+  Killed                  taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 0 > log.txt
root@IMX8MP-Tronlong:~#
```

图 23

3.1.2 SCHED_FIFO 模式（满负荷状态）

该测试状态为程序运行在隔离 CPU，程序进程调度策略设置为 SCHED_FIFO，进程优先级设置为最高，满负荷状态压力测试 12 个小时。

执行以下命令，控制两台伺服电机同时正转。

Target# stress --cpu 3 --io 3 --vm 3 -t 43200 &

Target# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 0 > log.txt &

```
root@IMX8MP-Tronlong:~# stress --cpu 3 --io 3 --vm 3 -t 43200 &
[1] 921
root@IMX8MP-Tronlong:~# stress-ng: info:  [921] setting to a 43200 second (12 hours, 0.00 secs) run per stressor
stress-ng: info:  [921] dispatching hogs: 3 cpu, 3 io, 3 vm

root@IMX8MP-Tronlong:~# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 0 > log.txt &
[2] 937
root@IMX8MP-Tronlong:~#
```

图 24

打印信息"937"为进程号，请根据实际进程号，执行如下命令，确认进程调度策略。

Target# chrt -p 937

```
root@IMX8MP-Tronlong:~# chrt -p 937
pid 937's current scheduling policy: SCHED_FIFO
pid 937's current scheduling priority: 99
root@IMX8MP-Tronlong:~#
```

图 25

如需停止运行后台程序，请执行如下命令。

Target# kill -9 937

```
root@IMX8MP-Tronlong:~# kill -9 937
root@IMX8MP-Tronlong:~#
[2]+  Killed                  taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 0 > log.txt
root@IMX8MP-Tronlong:~#
```

图 26

3.1.3 SCHED_DEADLINE 模式（空载状态）

该测试状态为程序运行在隔离 CPU，程序进程调度策略设置为 SCHED_DEADLINE、进程优先级设置为最高，空载状态测试 12 个小时。

执行如下命令，控制两台伺服电机同时正转。

Target# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 1 150000 1000000 1000000 > log.txt &

```
root@IMX8MP-Tronlong:~# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 1 150000 1000000 1000000 > log.txt &
[1] 945
root@IMX8MP-Tronlong:~#
```

图 27

打印信息"945"为进程号，请根据实际进程号，执行如下命令，确认进程调度策略。

Target# chrt -p 945

```
root@IMX8MP-Tronlong:~# chrt -p 945
pid 945's current scheduling policy: SCHED_DEADLINE
pid 945's current scheduling priority: 0
pid 945's current runtime/deadline/period parameters: 150000/1000000/1000000
root@IMX8MP-Tronlong:~#
```

图 28

如需停止运行后台程序，请执行如下命令。

Target# kill -9 945

```
root@IMX8MP-Tronlong:~# kill -9 945
root@IMX8MP-Tronlong:~#
[1]+  Killed                  taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 1 150000 1000000 1000000 > log.txt
root@IMX8MP-Tronlong:~#
```

图 29

3.1.4 SCHED_DEADLINE 模式（满负荷状态）

该测试状态为程序运行在隔离 CPU，进程调度策略设置为 SCHED_DEADLINE，优先级设置为最高，满负荷状态压力测试 12 个小时。

执行以下命令，控制两台伺服电机同时正转。

Target# stress --cpu 3 --io 3 --vm 3 -t 43200 &

Target# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 1 150000 1000000 1000000 > log.txt &

```
root@IMX8MP-Tronlong:~# stress --cpu 3 --io 3 --vm 3 -t 43200 &
[1] 950
root@IMX8MP-Tronlong:~# stress-ng: info:  [950] setting to a 43200 second (12 hours, 0.00 secs) run per stressor
stress-ng: info:  [950] dispatching hogs: 3 cpu, 3 io, 3 vm

root@IMX8MP-Tronlong:~# taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 1 150000 1000000 1000000 > log.txt &
[2] 963
root@IMX8MP-Tronlong:~#
```

图 30

打印信息"963"为进程号，请根据实际进程号，执行如下命令，确认进程调度策略。

Target# chrt -p 963

```
root@IMX8MP-Tronlong:~# chrt -p 963
pid 963's current scheduling policy: SCHED_DEADLINE
pid 963's current scheduling priority: 0
pid 963's current runtime/deadline/period parameters: 150000/1000000/1000000
root@IMX8MP-Tronlong:~#
```

图 31

如需停止运行后台程序，执行如下命令。

Target# kill -9 963

```
root@IMX8MP-Tronlong:~# kill -9 963
root@IMX8MP-Tronlong:~#
[2]+  Killed                  taskset -c 3 ./igh_ethercat_dc_motor -d 0 -s 1 150000 1000000 1000000 > log.txt
root@IMX8MP-Tronlong:~#
```

图 32

3.1.5 统计结果分析

程序运行输出的打印信息保存到了 log.txt，停止后台运行程序后，执行如下命令查看 log.txt 部分内容。

Target# head -n50 log.txt

```
root@IMX8MP-Tronlong:~# head -n50 log.txt
--> main: Request master success.
--> main: Create domain success.
--> main: Get slave configuration success.
--> main: Configuration PDO success.
--> main: success to register bunch of PDO entries for domain.
--> main: Activate master success.
--> main: Get pointer to the process data memory success.
--> main: Enter cycle task now...
--> check_master_state: 2 slave(s).
--> check_master_state: AL states: 0x02.
--> check_master_state: Link is up.
--> check_slave_config_state: slave[0]: State 0x02.
--> check_slave_config_state: slave[0]: online.
--> check_slave_config_state: slave[1]: State 0x02.
--> check_slave_config_state: slave[1]: online.
--> check_slave_config_state: slave[0]: State 0x01.
--> check_slave_config_state: slave[0]: State 0x02.
--> check_domain_state: Domain: WC 3.
--> check_domain_state: Domain: State 1.
--> check_slave_config_state: slave[0]: State 0x04.
--> check_slave_config_state: slave[0]: State 0x08.
--> check_slave_config_state: slave[0]: operational.
--> check_slave_config_state: slave[1]: State 0x01.
--> check_slave_config_state: slave[1]: State 0x02.
--> check_domain_state: Domain: WC 6.
--> check_domain_state: Domain: State 2.
--> check_slave_config_state: slave[1]: State 0x04.
--> check_slave_config_state: slave[1]: State 0x08.
--> check_slave_config_state: slave[1]: operational.
--> check_master_state: AL states: 0x08.
period     min:          0(ns) ... max: 4126524240(ns)
exec       min:          0(ns) ... max: 3831599680(ns)
latency    min:          0(ns) ... max:      32250(ns)
period     min:    1023875(ns) ... max:    1098125(ns)
exec       min:      15000(ns) ... max:      64750(ns)
latency    min:       8375(ns) ... max:      40750(ns)
period     min:    1023375(ns) ... max:    1085250(ns)
exec       min:      14750(ns) ... max:      67000(ns)
latency    min:       8500(ns) ... max:      25625(ns)
period     min:    1023625(ns) ... max:    1132375(ns)
exec       min:      14875(ns) ... max:      91875(ns)
latency    min:       8500(ns) ... max:      58375(ns)
period     min:    1023750(ns) ... max:    1100250(ns)
exec       min:      14875(ns) ... max:      52000(ns)
latency    min:       7500(ns) ... max:      51750(ns)
period     min:    1022000(ns) ... max:    1122500(ns)
exec       min:      14375(ns) ... max:      75500(ns)
latency    min:       7125(ns) ... max:      61750(ns)
period     min:    1023875(ns) ... max:    1094375(ns)
exec       min:      14625(ns) ... max:      77875(ns)
root@IMX8MP-Tronlong:~#
```

图 33

参数解释：

（1） latency：等待唤醒时间(ns)；

（2） period：EtherCAT 通讯周期时间(ns)；

（3） exec：接收和发送 EtherCAT 数据时间(ns)。

案例设置 period 为 1ms，通过分析 log.txt 得到 12 小时的 period 最小值和 period 最大值（period 单位：纳秒），通过分析计算 period 得到 12 个小时的 period 抖动值（period 最大值 - period 最小值），从 period 抖动值中分析、计算得到 period 最大抖动值、period 平均抖动值。

对 log.txt 文件内容进行数据分析后，得到如下结果。

表 2

进程调度模式+测试状态	最大抖动值（单位 ns）	平均抖动值（单位 ns）
SCHED_FIFO 模式（空载状态）	6546125	22733
SCHED_FIFO 模式（满负荷状态）	17826000	153784
SCHED_DEADLINE 模式（空载状态）	4165125	23240
SCHED_DEADLINE 模式（满负荷状态）	17612125	156869

备注：若进程调度模式 SCHED_FIFO 模式满足需求，推荐使用 SCHED_FIFO 模式，SCHED_DEADINE 模式在 CPU 高负载情况下可能出现丢包问题。

3.2 电机反转

请参考上述“电机正转”测试方法，将运行 igh_ethercat_dc_motor 可执行程序命令中的"-d 0"参数修改为"-d 1"。

4 案例编译

4.1 内核镜像、内核模块编译

请参考《Linux 系统使用手册》文档“配置内核编译选项”小节，打开图形配置界面，配置 fec 网卡驱动编译为内核模块，进入"Device Drivers -> Network device support ---> Ethernet driver support"路径，选择"EC ethernet controller (of ColdFire and some i.MX CPUs)"按下 M 键，会将相应的选项配置编译为内核模块，菜单选项前面变为**< M >**。

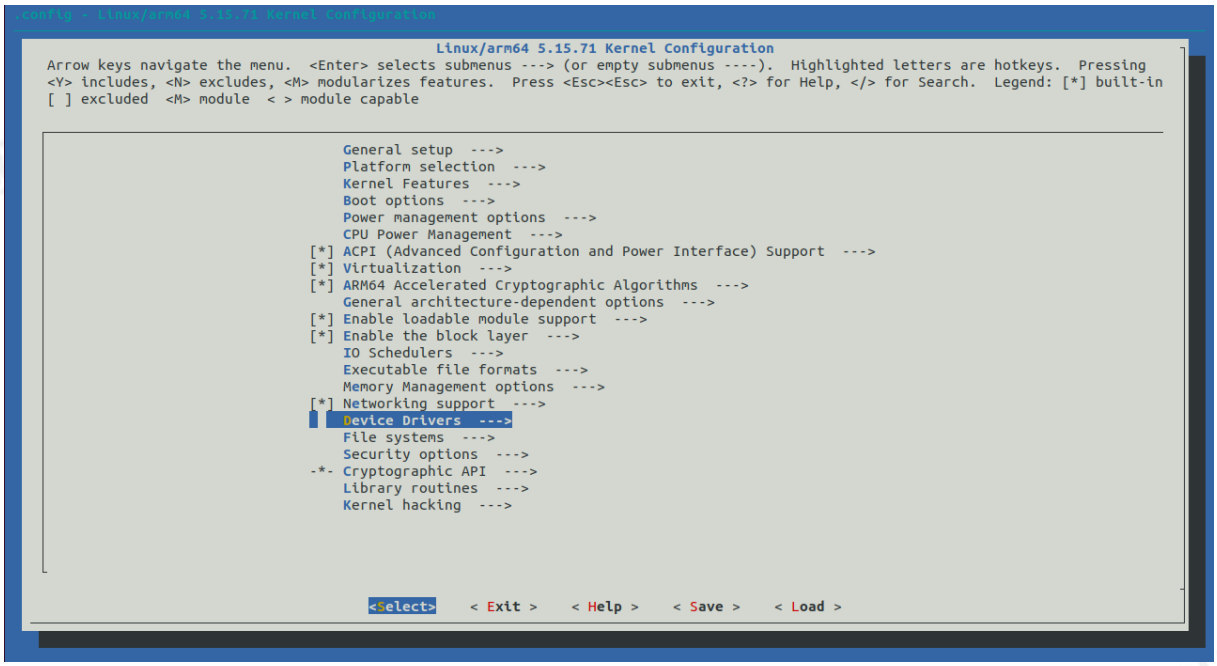

图 34

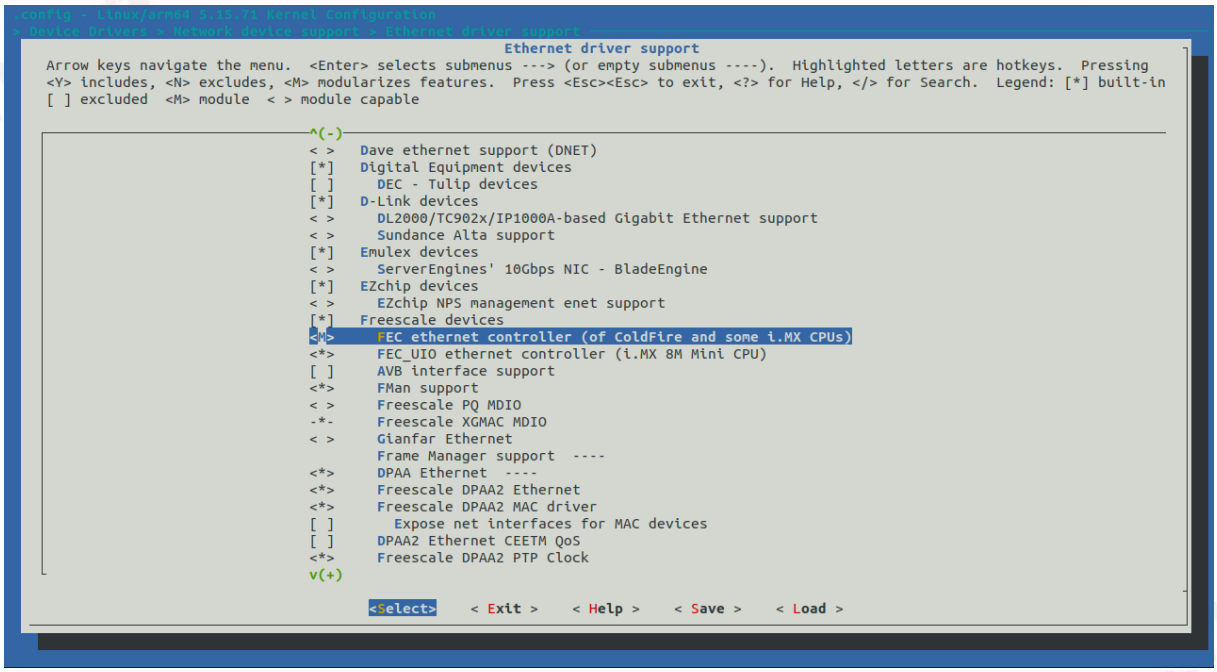

图 35

进入"General setup -> Preemption Model (Fully Preemptible Kernel (Real-Time))"路径，确保选择"Fully Preemptible Kernel (Real-Time)"。

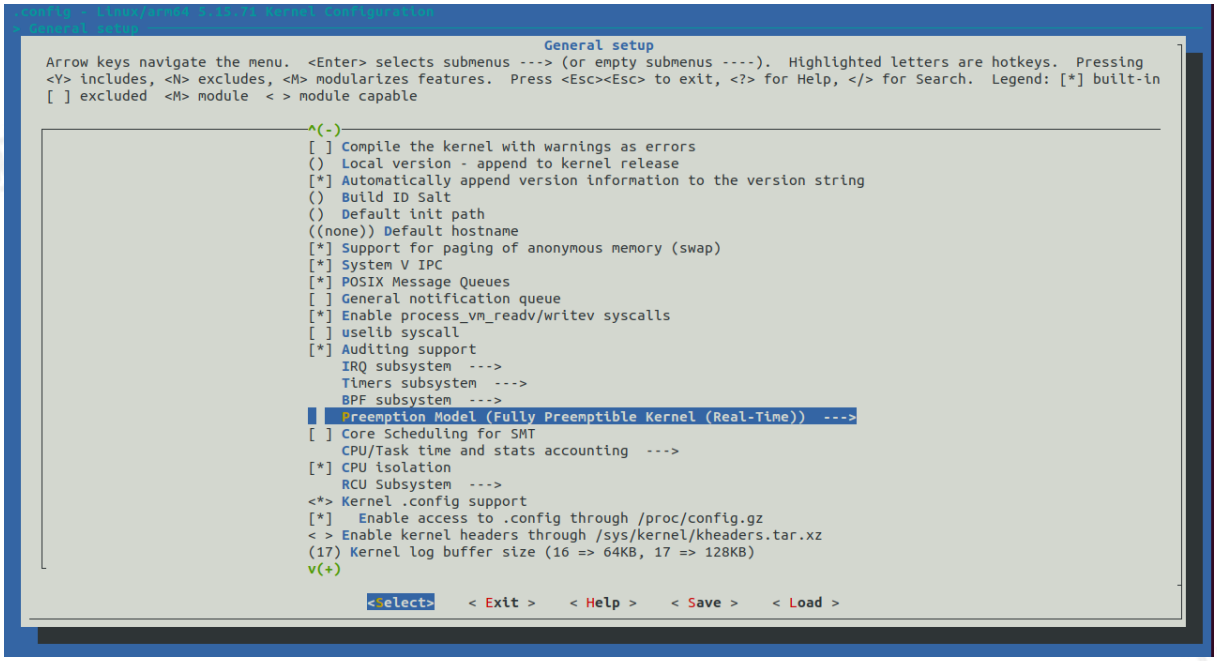

图 36

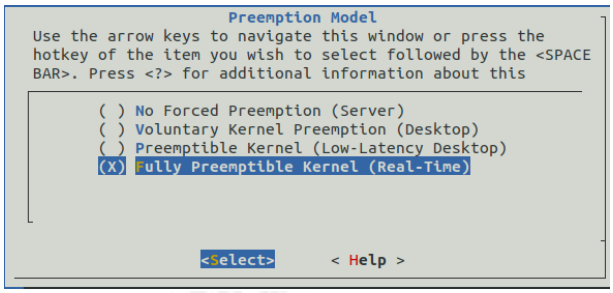

图 37

进入"Kernel Features -> Timer frequency(xxx HZ)"路径，选择 Timer frequency (1000 HZ)。

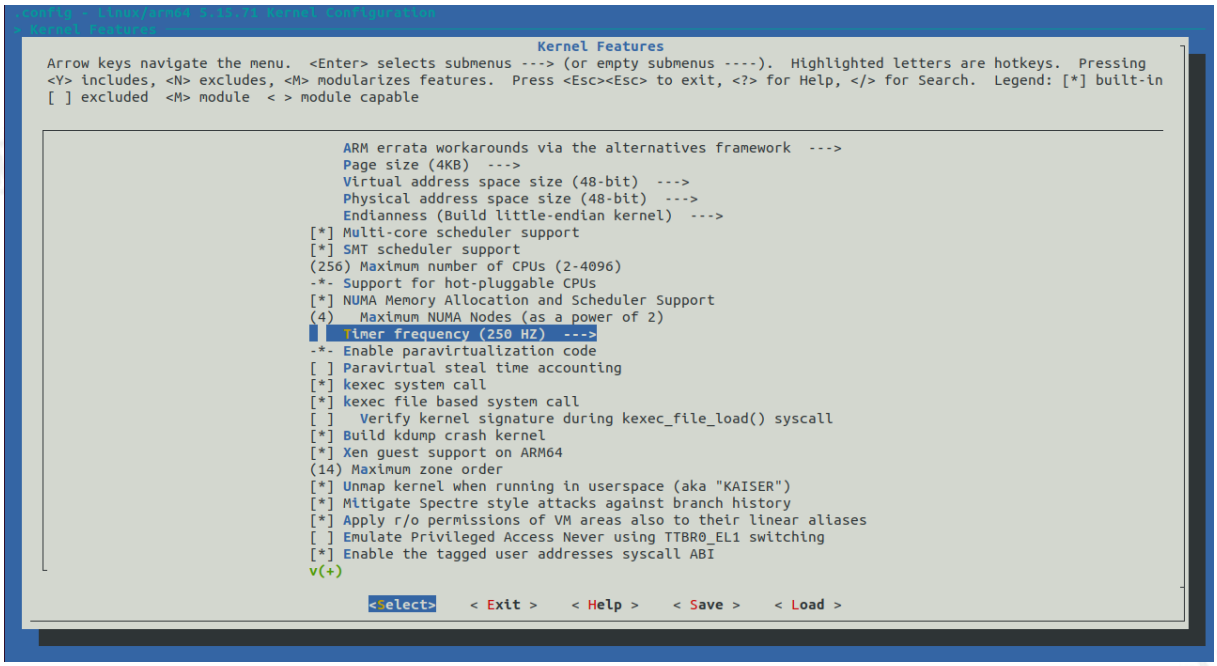

图 38

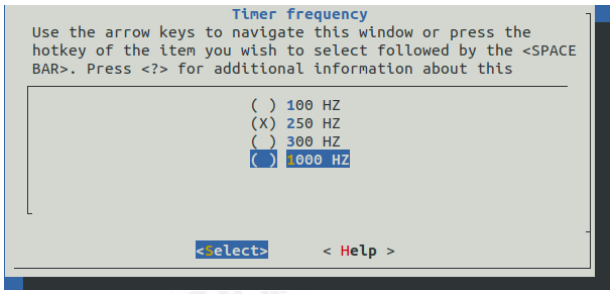

图 39

配置完成后，请参考《Linux 系统使用手册》，编译 Linux-RT 内核镜像、内核模块。

执行如下命令，新建内核模块安装文件夹。

Host# mkdir __install

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$ mkdir __install
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$
```

图 40

执行如下命令，安装内核模块至新建的文件夹。

Host# make modules_install INSTALL_MOD_PATH=/home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$ make modules_install INSTALL_MOD_PATH=/home/tronl
ong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/a
es-neon-blk.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/a
es-neon-bs.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/c
hacha-neon.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/c
rct10dif-ce.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/s
ha3-ce.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/s
ha512-arm64.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/s
ha512-ce.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/crypto/s
m3-ce.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/arch/arm64/lib/xor-
neon.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/af_alg.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/algif_aead.k
o
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/algif_hash.k
o
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/algif_rng.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/algif_skciph
er.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/anubis.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/arc4.ko
  INSTALL /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules/5.15.71-rt51/kernel/crypto/authenc.ko
```

图 41

执行如下命令，打包内核模块。

Host# cd __install/lib/modules/

Host# tar -czf 5.15.71-rt51.tar.gz 5.15.71-rt51/

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$ cd __install/lib/modules/
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules$ ls
5.15.71-rt51
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules$
```

图 42

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules$ tar -czf 5.15.71-rt51.tar.g
z 5.15.71-rt51/
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules$ ls
5.15.71-rt51  5.15.71-rt51.tar.gz
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51/__install/lib/modules$
```

图 43

4.2 Igh EtherCAT 主站程序编译

执行如下命令，创建 rt_linux_demos 目录。

Host# mkdir -p IMX8MP/rt_linux_demos/

```
tronlong@tronlong-virtual-machine:~$ mkdir -p IMX8MP/rt_linux_demos/
tronlong@tronlong-virtual-machine:~$ ls IMX8MP/rt_linux_demos/
tronlong@tronlong-virtual-machine:~$
```

图 44

将产品资料“4-软件资料\Demo\rt-linux-demos\”目录下的 igh_ethercat_dc_motor 文件夹拷贝至上述创建的目录。

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos$ ls
igh_ethercat_dc_motor
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos$ ls igh_ethercat_dc_motor/
bin  igh_ethercat  patch  src
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos$
```

图 45

执行如下命令，进入"igh_ethercat_dc_motor/igh_ethercat/src/"目录，将压缩包解压并进入解压后的文件夹。

Host# cd igh_ethercat_dc_motor/igh_ethercat/src/

Host# tar -xf ethercat-1.5.2-g418dc4b.tar.gz

Host# ls

Host# cd ethercat-1.5.2-g418dc4b/

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos$ cd igh_ethercat_dc_motor/igh_ethercat/src/
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src$ tar -xf ethercat-1.5.2-g418dc4b.tar.gz
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src$ ls
ethercat-1.5.2-g418dc4b  ethercat-1.5.2-g418dc4b.tar.gz
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src$ cd ethercat-1.5.2-g418dc4b/
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b$
```

图 46

执行如下命令，使能交叉编译工具链。

备注：请以实际 SDK 安装路径为准。

Host# source ~/SDK/environment-setup-armv8a-poky-linux

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ source ~/SDK/environment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$
```

图 47

执行如下命令，将所有补丁文件和 do_patch.sh 脚本拷贝至 ethercat-1.5.2-g418dc4b 目录，增加 do_patch.sh 脚本的可执行权限。

Host# cp ../../../patch/* ./

Host# chmod a+x do_patch.sh

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b$ cp ../../../patch/* ./
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b$ ls
0001-Add-support-for-NXP-DPAA1-ethercat-port.patch          0010-device-fec-add-fec-support-on-conf-script.patch        AUTHORS          ethercat.spec     Makefile.in
0001-Fixed-compilation-error-for-the-EtherCat-drivers.patch 0011-Fix-ethercat-tool-compilation.patch                autoconf         ethercat.spec.in  master
0002-igh_ethercat-fix-calltrace-issue-caused-by-ethercatc.patch 0012-enect-Add-enetc-souce-code-based-on-linux-4.14.patch ChangeLog       examples          NEWS
0002-Modify-the-example-code.patch                          0013-ec_enetc-Add-ec_enetc-native-driver-based-on-linux-4.patch config.h.in  FEATURES          README
0003-Fixed-compilation-error-for-the-IGH-EtherCAT.patch     0014-enetc-port-enetc-vf-driver-to-linux-5.10.patch         configure        globals.h         README.EoE
0004-replace-the-init_timer-with-timer_setup-function.patch 0015-enetc-add-pf-device-support-for-linux-5.10.patch     configure.ac     include           revision
0005-igh-ethernet-change-the-type-of-eccdev_vma_fault-to-.patch 0016-enetc-adjust-the-link-status-dynamically-for-ec_mast.patch COPYING  INSTALL  script
0006-examples-user-simplify-the-Igh-test-case.patch         0017-enetc-mq-is-useless-to-ndev-struct.patch               COPYING.LESSER   Kbuild.in         TODO
0007-configure-Fix-the-subdir-objects-error.patch           0018-enetc-add-enetc-support-on-scripts.patch               devices          lib               tool
0008-master-master-fix-the-issue-of-sched_setscheduler-un.patch 0019-devices-fec-fec_main-Fix-up-error-for-support-rt-lin.patch do_patch.sh  m4  tty
0009-device-fec-Add-fec-device.patch                        aclocal.m4                                                  Doxyfile.in      Makefile.am
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b$ chmod a+x do_patch.sh
```

图 48

执行如下命令，运行 do_patch.sh 脚本。

Host# source do_patch.sh

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ source do_patch.sh
patching file devices/generic.c
patching file master/ethernet.c
patching file examples/user/main.c
patching file master/cdev.c
patching file master/master.c
patching file examples/mini/mini.c
patching file master/cdev.c
patching file examples/user/main.c
patching file configure.ac
patching file master/master.c
patching file configure.ac
patching file devices/Kbuild.in
patching file devices/Makefile.am
patching file devices/fec/Kbuild.in
patching file devices/fec/Makefile.am
patching file devices/fec/fec.h
patching file devices/fec/fec_main.c
patching file devices/fec/fec_ptp.c
patching file script/ethercat.conf
patching file tool/Makefile.am
patching file tool/Makefile.in
patching file tool/soe_errors.c
patching file devices/enetc/enetc.c
patching file devices/enetc/enetc.h
patching file devices/enetc/enetc_cbdr.c
patching file devices/enetc/enetc_ethtool.c
patching file devices/enetc/enetc_hw.h
patching file devices/enetc/enetc_mdio.c
```

图 49

执行如下命令，重新生成配置文件。

Host# autoreconf -Wcross --verbose --install --force

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ autoreconf -Wcross --verbose --install --force
autoreconf: export WARNINGS=cross
autoreconf: Entering directory '.'
autoreconf: configure.ac: not using Gettext
autoreconf: running: aclocal --force -I m4
autoreconf: configure.ac: tracing
autoreconf: running: libtoolize --copy --force
libtoolize: putting auxiliary files in AC_CONFIG_AUX_DIR, 'autoconf'.
libtoolize: copying file 'autoconf/ltmain.sh'
libtoolize: putting macros in AC_CONFIG_MACRO_DIRS, 'm4'.
libtoolize: copying file 'm4/libtool.m4'
libtoolize: copying file 'm4/ltoptions.m4'
libtoolize: copying file 'm4/ltsugar.m4'
libtoolize: copying file 'm4/ltversion.m4'
libtoolize: copying file 'm4/lt~obsolete.m4'
autoreconf: configure.ac: not using Intltool
autoreconf: configure.ac: not using Gtkdoc
autoreconf: running: aclocal --force -I m4
autoreconf: running: /home/tronlong/SDK/sysroots/x86_64-pokysdk-linux/usr/bin/autoconf --force
configure.ac:57: warning: The macro `AC_PROG_LIBTOOL' is obsolete.
configure.ac:57: You should run autoupdate.
m4/libtool.m4:100: AC_PROG_LIBTOOL is expanded from...
configure.ac:57: the top level
configure.ac:64: warning: The macro `AC_HELP_STRING' is obsolete.
configure.ac:64: You should run autoupdate.
../autoconf-2.71/lib/autoconf/general.m4:204: AC_HELP_STRING is expanded from...
configure.ac:64: the top level
configure.ac:124: warning: The macro `AC_HELP_STRING' is obsolete.
configure.ac:124: You should run autoupdate.
../autoconf-2.71/lib/autoconf/general.m4:204: AC_HELP_STRING is expanded from...
configure.ac:124: the top level
```

图 50

```
configure.ac:524: warning: The macro `AC_HELP_STRING' is obsolete.
configure.ac:524: You should run autoupdate.
../autoconf-2.71/lib/autoconf/general.m4:204: AC_HELP_STRING is expanded from...
configure.ac:524: the top level
configure.ac:564: warning: The macro `AC_HELP_STRING' is obsolete.
configure.ac:564: You should run autoupdate.
../autoconf-2.71/lib/autoconf/general.m4:204: AC_HELP_STRING is expanded from...
configure.ac:564: the top level
configure.ac:612: warning: The macro `AC_HELP_STRING' is obsolete.
configure.ac:612: You should run autoupdate.
../autoconf-2.71/lib/autoconf/general.m4:204: AC_HELP_STRING is expanded from...
configure.ac:612: the top level
configure.ac:947: warning: The macro `AC_HELP_STRING' is obsolete.
configure.ac:947: You should run autoupdate.
../autoconf-2.71/lib/autoconf/general.m4:204: AC_HELP_STRING is expanded from...
configure.ac:947: the top level
autoreconf: running: /home/tronlong/SDK/sysroots/x86_64-pokysdk-linux/usr/bin/autoheader --force
autoreconf: running: automake --add-missing --copy --force-missing
configure.ac:55: installing 'autoconf/ar-lib'
configure.ac:54: installing 'autoconf/compile'
configure.ac:44: installing 'autoconf/missing'
examples/dc_user/Makefile.am: installing 'autoconf/depcomp'
autoreconf: Leaving directory '.'
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$
```

图 51

执行如下命令，创建__install 目录，并进行配置。

Host# mkdir -p __install

Host# ./configure --prefix=/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/__install --with-linux-dir=/home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/ --disable-8139too --enable-fec --enable-generic --host=aarch64-poky-linux

备注：--prefix 选择创建的__install 目录，--with-linux-dir 选择 Linux-RT 内核目录，Linux-RT 内核需先进行编译。

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ mkdir -p __install
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ ./configure --prefix=/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418
dc4b/__install --with-linux-dir=/home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/ --disable-8139too --enable-fec --enabl
e-generic --host=aarch64-poky-linux
configure: loading site script /home/tronlong/SDK/site-config-armv8a-poky-linux
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for aarch64-poky-linux-strip... aarch64-poky-linux-strip
checking for a race-free mkdir -p... /usr/bin/mkdir -p
checking for gawk... gawk
checking whether make sets $(MAKE)... yes
checking whether make supports nested variables... yes
checking whether make supports the include directive... yes (GNU style)
checking for aarch64-poky-linux-gcc... aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2
-D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-pok
y-linux
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... yes
checking for suffix of object files... o
checking whether the compiler supports GNU C... yes
checking whether aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -W
format -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-linux accepts -g...
yes
checking for aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -Wform
at -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-linux option to enable C
11 features... none needed
checking whether aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -W
format -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-linux understands -c
 and -o together... yes
```

图 52

```
config.status: creating lib/Makefile
config.status: creating m4/Makefile
config.status: creating master/Kbuild
config.status: creating master/Makefile
config.status: creating script/Makefile
config.status: creating script/ethercat.service
config.status: creating script/ethercatctl
config.status: creating script/init.d/Makefile
config.status: creating script/init.d/ethercat
config.status: creating script/sysconfig/Makefile
config.status: creating tool/Makefile
config.status: creating tty/Kbuild
config.status: creating tty/Makefile
config.status: creating config.h
config.status: executing depfiles commands
config.status: executing libtool commands
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$
```

图 53

执行如下命令，进行编译。

Host# make all modules

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ make all modules
make  all-recursive
make[1]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b'
Making all in devices
make[2]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices'
make  all-recursive
make[3]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices'
Making all in e1000
make[4]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices/e1000'
make  all-am
make[5]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices/e1000'
make[5]: Nothing to be done for 'all-am'.
make[5]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/devices/e1000'
make[4]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/devices/e1000'
Making all in e1000e
```

图 54

```
ec/fec_ptp.o
  LD [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/devices/f
ec/ec_fec.o
  CC [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/devices/g
eneric.o
  LD [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/devices/e
c_generic.o
  MODPOST /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/Module.sy
mvers
  CC [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/devices/e
c_generic.mod.o
  LD [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/devices/e
c_generic.ko
  CC [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/devices/f
ec/ec_fec.mod.o
  LD [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/devices/f
ec/ec_fec.ko
  CC [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/examples/
mini/ec_mini.mod.o
  LD [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/examples/
mini/ec_mini.ko
  CC [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/master/ec
_master.mod.o
  LD [M]  /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/master/ec
_master.ko
make[1]: Leaving directory '/home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51'
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$
```

图 55

执行如下命令，进行安装。

Host# make install systemdsystemunitdir=/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4b/__install

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ make install systemdsystemunitdir=/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/etherc
at-1.5.2-g418dc4b/__install
make  install-recursive
make[1]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b'
Making install in devices
make[2]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices'
make  install-recursive
make[3]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices'
Making install in e1000
make[4]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices/e1000'
make  install-am
make[5]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices/e1000'
make[6]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b/devices/e1000'
make[6]: Nothing to be done for 'install-exec-am'.
make[6]: Nothing to be done for 'install-data-am'.
make[6]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/devices/e1000'
make[5]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/devices/e1000'
make[4]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/devices/e1000'
Making install in e1000e
```

图 56

```
make[4]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/examples'
make[3]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/examples'
make[2]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b/examples'
make[2]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b'
make[3]: Entering directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2
-g418dc4b'
make[3]: Nothing to be done for 'install-exec-am'.
make[3]: Nothing to be done for 'install-data-am'.
make[3]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b'
make[2]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b'
make[1]: Leaving directory '/home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-
g418dc4b'
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$
```

图 57

执行如下命令，创建 modules 目录，并将.ko 文件拷贝至该目录。

Host# mkdir -p __install/modules

Host# cp master/ec_master.ko __install/modules/

Host# cp devices/ec_generic.ko __install/modules/

Host# cp devices/fec/ec_fec.ko __install/modules/

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ mkdir -p __install/modules
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ cp master/ec_master.ko __install/modules/
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ cp devices/ec_generic.ko __install/modules/
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$ cp devices/fec/ec_fec.ko __install/modules/
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2-g418dc4
b$
```

图 58

4.3 igh_ethercat_dc_motor 案例编译

igh_ethercat_dc_motor 案例的编译需基于 IgH EtherCAT 主站程序，请先编译 IgH EtherCAT 主站程序。

请将案例 src 源码目录拷贝至 Ubuntu 工作目录下，进入 src 目录执行如下命令修改 Makefile 相关内容为 IgH EtherCAT 主站程序对应目录，如下图所示。

Host# vim Makefile

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/src$ ls
igh_ethercat_dc_motor.c  Makefile  parameter_parser.c  parameter_parser.h
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/src$ vim Makefile
```

图 59

```
CFLAGS  += -g -Wall -O2 $(DEFINES) $(INCLUDE)
LIBS    += -lethercat
CXXFLAGS:= $(CFLAGS)
SOURCE  := $(wildcard *.c) $(wildcard *.cpp)
OBJS    := $(patsubst %.c,%.o,$(patsubst %.cpp,%.o,$(SOURCE)))
TARGET  := igh_ethercat_dc_motor
MINCLUDE := -I /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2/__install/include
LDFLAGS := -L /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_ethercat/src/ethercat-1.5.2/__install/lib

.PHONY : everything objs clean distclean rebuild

all : $(TARGET)

objs : $(OBJS)

rebuild: distclean all

clean :
        rm -rf *~
        rm -rf *.o

distclean : clean
        rm -rf $(TARGET)

$(TARGET) : $(OBJS)
        $(CC) $(CXXFLAGS) -o $@ $(OBJS) $(LDFLAGS) $(LIBS)
```

图 60

执行如下命令，进行编译。编译完成后，将在当前目录下生成可执行文件。

Host# source ~/SDK/environment-setup-armv8a-poky-linux //使能交叉编译工具链

Host# make

```
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/src$ source ~/SDK/environment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=f
ormat-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall -O2     -c
 -o igh_ethercat_dc_motor.o igh_ethercat_dc_motor.c
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=f
ormat-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall -O2     -c
 -o parameter_parser.o parameter_parser.c
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=f
ormat-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall -O2   -o i
gh_ethercat_dc_motor igh_ethercat_dc_motor.o parameter_parser.o -L /home/tronlong/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/igh_etherc
at/src/ethercat-1.5.2/__install/lib -lethercat
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/src$ ls
igh_ethercat_dc_motor    igh_ethercat_dc_motor.o  parameter_parser.c  parameter_parser.o
igh_ethercat_dc_motor.c  Makefile                 parameter_parser.h
tronlong@tronlong-virtual-machine:~/IMX8MP/rt_linux_demos/igh_ethercat_dc_motor/src$
```

图 61

5 关键代码说明

（1） 创建 Ethercat master。

```
/* Request EtherCAT master */
master = ecrt_request_master(0);
if (!master) {
    printf("--> main: Request master failed.\n");
    return -1;
}
printf("--> main: Request master success.\n");
```

图 62

（2） 创建域。

```
/* Create domain */
domain.domain = ecrt_master_create_domain(master);
if (!domain.domain) {
    status = -1;
    printf("--> main: Create domain failed.\n");
    goto err_leave;
}
printf("--> main: Create domain success.\n");
```

图 63

（3） 配置 PDO。

```
/* Get slave configuration */
for (i = 0; i < SLAVE_NUM; i++) {
    slave_config[i].sc = ecrt_master_slave_config(master, slave_info[i].alias,
                         slave_info[i].position, slave_info[i].vendor_id,
                         slave_info[i].product_code);
    if (!slave_config[i].sc) {
        status = -1;
        printf("--> main: Get slave configuration failed.\n");
        goto err_leave;
    }
}
printf("--> main: Get slave configuration success.\n");

/* Configuring PDO */
for (i = 0; i < SLAVE_NUM; i++) {
    ret = ecrt_slave_config_pdos(slave_config[i].sc, EC_END, slave_syncs);
    if (ret != 0) {
        status = -1;
        printf("--> main: Configuration PDO failed.\n");
        goto err_leave;
    }
}
printf("--> main: Configuration PDO success.\n");
```

图 64

（4） 为域注册 PDO 条目。

```
/* Registers a bunch of PDO entries for a domain. */
ret = ecrt_domain_reg_pdo_entry_list(domain.domain, domain_regs);
if (ret != 0) {
    status = -1;
    printf("--> main: Failed to register bunch of PDO entries for domain.\n");
    goto err_leave;
}
printf("--> main: success to register bunch of PDO entries for domain.\n");
```

图 65

（5） 激活 master。

```
/* Activate EtherCAT master */
ret = ecrt_master_activate(master);
if (ret < 0) {
    status = -1;
    printf("--> main: Activate master failed.\n");
    goto err_leave;
}
printf("--> main: Activate master success.\n");
```

图 66

（6） 设置进程调度策略。

```
540     if(params.mode){
541         if(sched_setattr(pid, &attr, 0)){
542             printf("sched_setattr error!!\n");
543             goto err_leave;
544         }
545     }else{
546         /* The process priority is set to the maximum */
547         param.sched_priority = sched_get_priority_max(SCHED_FIFO);
548         if(sched_setscheduler(pid, SCHED_FIFO, &param)){
549             printf("sched_setscheduler error!!\n");
550             goto err_leave;
551         }
552     }
```

图 67

（7） 进入循环周期任务。

```
554     /* Start cyclic function. */
555     printf("--> main: Enter cycle task now...\n");
556     cyclic_task(slave_config, &domain, params.direction);
```

图 68

（8） 周期任务。

```
382     for (i = SLAVE_NUM - 1; i >= 0; i--) {
383         /* Read state word */
384         status = EC_READ_U16(domain->domain_pd + slave_offset[i].status_word);
385         if ((status & command[i]) == 0x0040) {
386             /* Set operation mode to velocity Mode */
387             EC_WRITE_U8(domain->domain_pd + slave_offset[i].operation_mode, MOTOR_MODEL_PROFILE_POSITION);
388             /* Set acceleration and  deceleration */
389             EC_WRITE_U32(domain->domain_pd + slave_offset[i].acceleration, MOTOR_PROFILE_ACCELERATION);
390             EC_WRITE_U32(domain->domain_pd + slave_offset[i].deceleration, MOTOR_PROFILE_DECELERATION);
391             EC_WRITE_U16(domain->domain_pd + slave_offset[i].ctrl_word, 0x0006);
392             command[i] = 0x006F;
393         } else if ((status & command[i]) == 0x0021) {
394             EC_WRITE_U16(domain->domain_pd + slave_offset[i].ctrl_word, 0x0007);
395             command[i] = 0x006F;
396         } else if ((status & command[i]) == 0x0023) {
397             EC_WRITE_U16(domain->domain_pd + slave_offset[i].ctrl_word, 0x000f);
398             command[i] = 0x006F;
```

图 69

（9） 发送目标速度，让驱动器开始转动。

```
399                 } else if ((status & command[i]) == 0x0027) {
400                     /* Get Internal Speed */
401                     slave_config[i].cur_speed = EC_READ_U32(domain->domain_pd + slave_offset[i].current_speed);
402                     /* Determine whether to turn forward or reverse */
403                     if (direction == 0) {
404                         /* forward */
405                         if (slave_config[i].cur_speed == 0) {
406                             EC_WRITE_S32(domain->domain_pd + slave_offset[i].target_speed, MOTOR_TARGET_SPEED_FORWARD);
407                         } else if(slave_config[i].cur_speed == MOTOR_TARGET_SPEED_FORWARD) {
408                             EC_WRITE_S32(domain->domain_pd + slave_offset[i].target_speed, 0);
409                         }
410                     } else if (direction == 1) {
411                         /* reverse */
412                         if (slave_config[i].cur_speed == 0) {
413                             EC_WRITE_S32(domain->domain_pd + slave_offset[i].target_speed, MOTOR_TARGET_SPEED_REVERSE);
414                         } else if (slave_config[i].cur_speed == MOTOR_TARGET_SPEED_REVERSE) {
415                             EC_WRITE_S32(domain->domain_pd + slave_offset[i].target_speed, 0);
416                         }
417                     }
```

图 70

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com